

FATIMA JINNAH WOMEN UNIVERSITY, RAWALPINDI



Group members:

Almas Shafqat (2023-BCS-006)

Romaisa (2023-BCS-073)

Sameen Noor (2023-BCS-077)

Course: Parallel & Distributing Computing

Instructor: Mam Madiha Wahid

Section: BCS-6(B)

Table of Contents

1. Introduction.....	3
2. Apache Airflow Overview	3
3. Project Overview.....	3
4. Pipeline 1 — Student Data ETL	4
5. Pipeline 2 — Employee Multi-File ETL	6
6. Scheduling and Dependency Management.....	8
7. Monitoring with Airflow Dashboard	9
8. Comparison of Both Pipelines	10
9. Tools and Technologies Used.....	10
10. Conclusion	11

Project Report

Workflow Automation Using Apache Airflow

1. Introduction

In today's data-driven world, enormous volumes of information are generated every second from diverse sources including databases, sensors, APIs, and user interactions. Managing and processing this data manually is not only time-consuming but also prone to human error. Workflow automation tools have emerged as a critical solution to this challenge, enabling organisations to build reliable, repeatable, and scalable data pipelines.

Apache Airflow is one of the most widely adopted open-source platforms for this purpose. In this project, we leverage Apache Airflow to design and implement two fully automated ETL (Extract, Transform, Load) data pipelines — one for student academic records and another for multi-department employee salary data. Both pipelines demonstrate real-world automation patterns including scheduling, dependency management, error handling, and live monitoring.

2. Apache Airflow Overview

Apache Airflow is an open-source workflow management platform that allows users to programmatically author, schedule, and monitor workflows. It uses Python for defining workflows, making it highly flexible and developer-friendly.

The core concept in Airflow is the **Directed Acyclic Graph (DAG)**. A DAG represents a workflow as a series of tasks connected in a specific order. Each task runs only after its dependencies are completed, ensuring proper execution flow.

Key Features

- **Task Scheduling:** Automates workflows at specified times (e.g., daily at 9 AM)
- **Dependency Management:** Ensures tasks run in the correct order
- **Retry Mechanism:** Automatically retries failed tasks
- **Monitoring Dashboard:** Provides real-time visualization of workflows

3. Project Overview

This project consists of two ETL pipelines, both following the same three-step process:

Extract → Transform → Load

However, they differ in terms of data sources, transformations, and outputs.

Feature	Pipeline 1: Student ETL	Pipeline 2: Employee ETL
Data Source	Single CSV file (data.csv)	Three CSV files (HR, IT, Finance)
Transformation	Grading, pass/fail, performance labels	Bonus calculation, salary categories, sorting
Output	SQLite database (etl.db)	Final CSV file (final_employees.csv)
Schedule	Daily at 9:00 AM	Daily at 9:00 AM
DAG ID	student_etl_pipeline	employee_multi_file_etl

4. Pipeline 1 — Student Data ETL

4.1 Purpose

This pipeline processes student academic records. It reads raw data, calculates grades and performance indicators, and stores the final processed data in a database.

4.2 Extract Phase

The extract step reads data from a CSV file (data.csv). A copy of this data is saved as extracted.csv to ensure that the original dataset remains unchanged. This step ensures data integrity and provides a clean input for further processing.

4.3 Transform Phase

This is the most critical stage where raw data is processed and enhanced. The following operations are performed:

- Student names are converted to uppercase
- Grades are assigned based on marks:
 - A (85 and above)
 - B (70–84)
 - C (50–69)
 - F (below 50)
- Pass/Fail status is determined
- Performance categories are assigned (Excellent, Good, Average, Poor)
- Only students who pass are retained

- Average marks are calculated and displayed in logs

This step ensures that the data becomes meaningful and ready for analysis.

```
AIRFLOW_CTX_DAG_OWNER=romaisa
AIRFLOW_CTX_DAG_ID=student_etl_pipeline
AIRFLOW_CTX_TASK_ID=transform_task
AIRFLOW_CTX_EXECUTION_DATE=2026-04-24T09:00:00+00:00
AIRFLOW_CTX_TRY_NUMBER=1
AIRFLOW_CTX_DAG_RUN_ID=scheduled__2026-04-24T09:00:00+00:00
[2026-04-26, 08:54:14 UTC] {logging_mixin.py:120} INFO - Starting Transform Task...
[2026-04-26, 08:54:14 UTC] {logging_mixin.py:120} INFO - Average Marks: 69.2
[2026-04-26, 08:54:14 UTC] {logging_mixin.py:120} INFO - Transform Completed!
```

4.4 Load Phase

The processed data is stored in a SQLite database (etl.db). A table named students is created or replaced with updated data. This ensures that the latest processed data is always available.

Additionally, the number of records inserted is logged for verification.

```
PS C:\Users\romai\materials> docker exec -it materials-airflow-worker-1 sqlite3
etl.db "SELECT * FROM students;"
id  name  age  marks  city      grade  status  performance
--  ---  --  -
1   ALI   22   78     Lahore    B      Pass    Good
2   SARA  25   88     Karachi   A      Pass    Excellent
4   AYESHA 21   92     Lahore    A      Pass    Excellent
5   USMAN 23   67     Multan    C      Pass    Average
6   ZARA  24   81     Karachi   B      Pass    Good
8   NOOR  22   73     Islamabad B      Pass    Good
8   NOOR  22   73     Islamabad B      Pass    Good
9   FAIZA 27   55     Multan    C      Pass    Average
10  BILAL 29   90     Karachi   A      Pass    Excellent
12  SAAD  23   76     Karachi   B      Pass    Good
13  HINA  24   84     Islamabad B      Pass    Good
14  TALHA 25   62     Multan    C      Pass    Average
15  MEHAK 22   91     Lahore    A      Pass    Excellent
16  REHAN 27   53     Karachi   C      Pass    Average
17  IQRA  21   69     Islamabad C      Pass    Average
19  MARIA 23   87     Lahore    A      Pass    Excellent
20  ASAD  28   72     Karachi   B      Pass    Good
```

4.5 DAG Structure

The workflow follows a simple sequential structure:

extract_task → transform_task → load_task

This ensures that each step runs only after the previous one completes successfully.



5. Pipeline 2 — Employee Multi-File ETL

5.1 Purpose

This pipeline processes employee data collected from multiple departments. It combines datasets, performs salary-based transformations, and produces a final structured output file.

5.2 Extract Phase

Three CSV files are used as input:

- HR department
- IT department
- Finance department

Each dataset is tagged with a new column called department. All files are merged into a single dataset (extracted.csv).

hr.csv:

```
dags > hr.csv > data
1 id,name,age,salary
2 1,Ali,25,50000
3 2,Sara,28,60000
4 3,Ahmed,30,55000
5 4,Ayesha,27,52000
6 5,Usman,29,58000
```

finance.csv

```
dags > finance.csv > data
1 id,name,age,salary
2 11,Amna,27,68000
3 12,Saad,30,71000
4 13,Hina,26,69000
5 14,Talha,28,67000
6 15,Mehak,25,66000
```

it.csv:

```
dags > it.csv > data
1 id,name,age,salary
2 6,Zara,26,70000
3 7,Hassan,31,75000
4 8,Noor,24,65000
5 9,Faiza,29,72000
6 10,Bilal,32,80000
```

5.3 Transform Phase

The following operations are applied:

- Employee names are converted to uppercase
- A 10% bonus is calculated
- Total salary is computed (salary + bonus)
- Employees are categorized:
 - High ($\geq 75,000$)
 - Medium (60,000–74,999)
 - Low ($< 60,000$)
- Low-category employees are removed
- Remaining data is sorted by total salary (descending)

This transformation improves data quality and focuses only on relevant records.

5.4 Load Phase

The fully processed dataset is written to the final output file `final_employees.csv`. The number of records exported is printed to the Airflow task logs so operators can quickly verify the volume of data without opening the file.

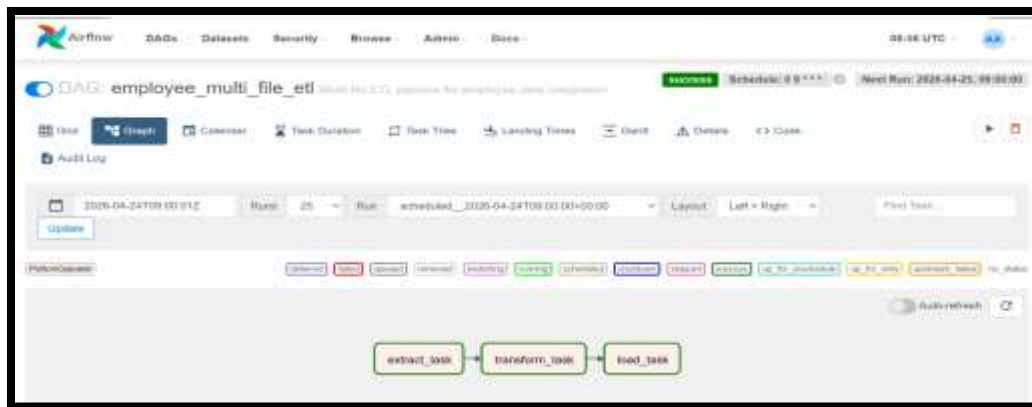
A	B	C	D	E	F	G	H
id	name	age	salary	department	bonus	total_salary	category
10	BILAL	32	80000	IT	8000	88000	High
7	HASSAN	31	75000	IT	7500	82500	High
9	FAIZA	29	72000	IT	7200	79200	Medium
12	SAAD	30	71000	Finance	7100	78100	Medium
6	ZARA	26	70000	IT	7000	77000	Medium
13	HINA	26	69000	Finance	6900	75900	Medium
11	AMNA	27	68000	Finance	6800	74800	Medium
14	TALHA	28	67000	Finance	6700	73700	Medium
15	MEHAK	25	66000	Finance	6600	72600	Medium
8	NOOR	24	65000	IT	6500	71500	Medium
2	SARA	28	60000	HR	6000	66000	Medium

5.5 DAG Structure

Identical to Pipeline 1, the employee pipeline uses a simple sequential chain:

extract_task → transform_task → load_task

The linear dependency ensures data integrity at every stage: the transform task never operates on partially written extraction output, and the load task always receives a complete and validated transformation result.



6. Scheduling and Dependency Management

Both pipelines are set to run automatically using a cron schedule. The following line is defined in each DAG:

```
schedule_interval = '0 9 * * *'
```

This means the pipelines execute daily at 9:00 AM, without needing any manual trigger.



6.1 Catch-up Behavior

catchup = False is used in both DAGs. This stops Airflow from running old missed tasks when it restarts.

This is useful because only the latest data run is needed, not past executions.

6.2 Fault Tolerance Configuration

The following error-handling defaults are applied to all tasks in both DAGs:

- **retries: 2**
A failing task will be automatically retried up to two additional times before it is marked as failed.
- **retry_delay: 2 minutes**
Airflow waits two minutes between each retry attempt, giving transient issues time to resolve.
- **execution_timeout: 5 minutes**
Any task that exceeds five minutes of runtime is automatically terminated to prevent pipeline stalls

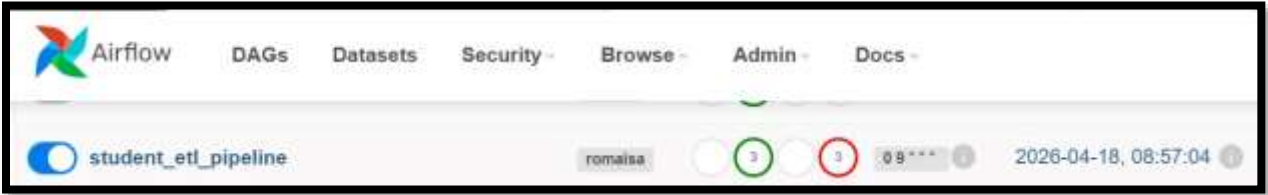
7. Monitoring with Airflow Dashboard

Apache Airflow comes with a built-in web interface that can be used to monitor all pipelines. This dashboard gives a clear picture of what is running, what has completed, and what has failed.

Key monitoring features we used:

- **DAG Graph View** — shows the flow of tasks and highlights which ones succeeded or failed using color coding (green = success, red = failed)
- **Task Logs** — each task records what it is doing in real time, so we can see print messages like 'Extract Completed!' directly in the logs

- **Run History** — shows a list of all past runs with their start time, end time, and status
- **Task Duration** — shows how long each task took, which helps find slow tasks



8. Comparison of Both Pipelines

The table below provides a side-by-side comparison of the two pipelines, highlighting the similarities in architecture and the differences in data handling and output.

Aspect	Student ETL	Employee ETL
Input	1 CSV file	3 CSV files (merged)
Main Logic	Grading & filtering students	Salary calculations & filtering
Output Type	SQLite database	CSV file
Records Kept	Only passing students	Only medium and high earners
Extra Feature	Grade & performance labels	Bonus and total salary columns

9. Tools and Technologies Used

Tool / Technology	Purpose
Apache Airflow	Building, scheduling, and monitoring the data pipelines
Python	Writing all task logic and data processing code

Tool / Technology	Purpose
Pandas	Reading CSV files and performing data transformations
SQLite	Storing the final student data in a local database (Pipeline 1)
Docker	Running Airflow in an isolated container environment
CSV Files	Input and output data format for both pipelines

10. Conclusion

This project successfully demonstrates the implementation of automated ETL pipelines using Apache Airflow. By designing two distinct pipelines, we explored how workflow automation can handle different real-world data scenarios efficiently.

The student pipeline focuses on academic data processing, while the employee pipeline handles multi-source salary data. Both pipelines highlight the importance of structured workflows, dependency management, and automation.

Airflow's scheduling, retry mechanisms, and monitoring dashboard significantly enhance reliability and efficiency. This project provided valuable insights into modern data engineering practices and reinforced the importance of automation in handling large-scale data workflows.